COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 / Data Structures and Algorithms	**SECTION:**	CPE21S4
DATE PERFORMED:	September 9-16, 2026	**DATE SUBMITTED:**	September 16, 2026
NAME:	DE VERA, Sophia D.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY 8.1 SORTING ALGORITHMS PT2

1. PROCEDURE OUTPUT

main.cpp > ...

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include "SMQ.h"


const int SIZE = 100;

void displayArray(int array[], int size){
    for (int i = 0; i < size; i++){
        std::cout << array[i] << " ";

        if ((i + 1) % 10 == 0){
            std::cout << std::endl;
        }
    }
}
```

```cpp
int main()
{
    int originalArray[SIZE];
    int shellArray[SIZE];
    int mergeArray[SIZE];
    int quickArray[SIZE];

    std::srand(time(0));

    // Create 100 random elements
    for (int i = 0; i < SIZE; i++)
    {
        originalArray[i] = rand() % 1000 + 1;

        shellArray[i] = originalArray[i];
        mergeArray[i] = originalArray[i];
        quickArray[i] = originalArray[i];
    }
```

SMQ.h > quickSort(T [], int, int)

```cpp
#ifndef SMQ_h
#define SMQ_h

template <typename T>
class SortingAlgorithms{
    private:
        // Merge two sorted portions of the array
        void merge(T array[], int left, int middle, int right);
        // Pivot the array for Quick Sort
        int pivot(T array[], int low, int high);
    public:
        // Shell Sort
        void shellSort(T array[], int size);
        // Merge Sort
        void mergeSort(T array[], int left, int right);
        // Quick Sort
        void quickSort(T array[], int low, int high);
};
```

```cpp
    SortingAlgorithms<int> sorting;

    std::cout << "THE 100 UNSORTED ELEMENTS: " << std::endl;
    std::cout << std::endl;
    displayArray(originalArray, SIZE);

    std::cout << std::endl;
```

```
THE 100 UNSORTED ELEMENTS:

539 345 501 557 925 822 58 669 654 384
699 105 251 466 483 194 743 68 919 665
37 891 489 238 301 971 606 90 214 498
426 898 158 14 179 951 670 2 891 580
858 285 738 24 839 484 547 957 621 561
21 576 479 467 901 106 776 624 75 510
225 89 642 980 593 909 92 917 92 136
677 576 759 773 510 645 378 338 198 136
942 339 884 735 666 273 71 664 233 493
329 189 382 176 634 89 123 612 289 432
```

COMPUTER ENGINEERING

QUEZON CITY

SHELL SORT TECHNIQUE

SCREENSHOT

```
45
46      sorting.shellSort(shellArray, SIZE);
47
48      std::cout << "SHELL SORTED ARRAY: " << std::endl;
49      std::cout << std::endl;
50      displayArray(shellArray, SIZE);
51
52      std::cout << std::endl;
```

```
21  // Shell Sort
22  template <typename T>
23  void SortingAlgorithms<T>::shellSort(T array[], int size){
24      for (int interval = size / 2; interval > 0; interval /= 2){
25          for (int i = interval; i < size; i++){
26              T temp = array[i];
27              int j = i;
28
29              while (j >= interval && array[j - interval] > temp){
30                  array[j] = array[j - interval];
31                  j = j - interval;
32              }
33              array[j] = temp;
34          }
35      }
36  }
```

```
SHELL SORTED ARRAY:

2 14 21 24 37 58 68 71 75 89
89 90 92 92 105 106 123 136 136 158
176 179 189 194 198 214 225 233 238 251
273 285 289 301 329 338 339 345 378 382
384 426 432 466 467 479 483 484 489 493
498 501 510 510 539 547 557 561 576 576
580 593 606 612 621 624 634 642 645 654
664 665 666 669 670 677 699 735 738 743
759 773 776 822 839 858 884 891 891 898
901 909 917 919 925 942 951 957 971 980
```

OBSERVATION ANALYSIS

- So since shell sort is an improved version technique of Insertion Sort, the program compares elements. The program uses a two **for** loops. The first for loop is functioned to control the interval of the code. While on the other hand, the second loop is used for the current interval (since we loop with big gaps with each other.) Then the **while** is for comparing the current element and the element in the **temp** value.

COMPUTER ENGINEERING

QUEZON CITY

MERGE SORT TECHNIQUE

SCREENSHOT

```cpp
// Merge Sort - Merge Function
template <typename T>
void SortingAlgorithms<T>::merge(T array[], int left, int middle, int right){
    int leftSize = middle - left + 1;
    int rightSize = right - middle;

    T* leftArray = new T[leftSize];
    T* rightArray = new T[rightSize];

    for (int i = 0; i < leftSize; i++){
        leftArray[i] = array[left + i];
    }

    for (int j = 0; j < rightSize; j++){
        rightArray[j] = array[middle + 1 + j];
    }

    while (i < leftSize && j < rightSize){
        if (leftArray[i] <= rightArray[j]){
            array[k] = leftArray[i];
            i++;
        } else {
            array[k] = rightArray[j];
            j++;
        }
        k++;
    }

    while (i < leftSize){
        array[k] = leftArray[i];
        i++;
        k++;
    }

    while (j < rightSize){
        array[k] = rightArray[j];
        j++;
        k++;
    }

    delete[] leftArray;
    delete[] rightArray;
}
```

```cpp
// Merge Sort
template <typename T>
void SortingAlgorithms<T>::mergeSort(T array[], int left, int right){
    if (left < right){
        int mid = left + (right - left) / 2;

        mergeSort(array, left, mid);
        mergeSort(array, mid + 1, right);

        merge(array, left, mid, right);
    }
}
```

```
MERGE SORTED ARRAY:

2 14 21 24 37 58 68 71 75 89
89 90 92 92 105 106 123 136 136 158
176 179 189 194 198 214 225 233 238 251
273 285 289 301 329 338 339 345 378 382
384 426 432 466 467 479 483 484 489 493
498 501 510 510 539 547 557 561 576 576
580 593 606 612 621 624 634 642 645 654
664 665 666 669 670 677 699 735 738 743
759 773 776 822 839 858 884 891 891 898
901 909 917 919 925 942 951 957 971 980
```

OBSERVATION ANALYSIS

- The merge sort is working into smaller sections. It first checks the current element (the j) and compares it to the previous element (the i index). Then the middle variable finds the center of the section, and the function calls itself recursively to sort the left and right portions of the array.

QUICK SORT TECHNIQUE

SCREENSHOT

```
// Quick Sort - Pivot Function
template <typename T>
int SortingAlgorithms<T>::pivot(T array[], int low, int high){
    T pivot = array[high];

    int i = low - 1;

    for (int j = low; j < high; j++){
        if (array[j] < pivot){
            i++;

            T temp = array[i];
            array[i] = array[j];
            array[j] = temp;
        }
    }

    T temp = array[i + 1];
    array[i + 1] = array[high];
    array[high] = temp;

    return i + 1;
}


// Quick Sort
template <typename T>
void SortingAlgorithms<T>::quickSort(T array[], int low, int high){
    if (low < high){
        int pivotIndex = pivot(array, low, high);

        quickSort(array, low, pivotIndex - 1);
        quickSort(array, pivotIndex + 1, high);
    }
}
```

```
    sorting.quickSort(quickArray, 0, SIZE - 1);

    std::cout << "QUICK SORTED ARRAY: " << std::endl;
    std::cout << std::endl;
    displayArray(quickArray, SIZE);

    return 0;
}
```

```
QUICK SORTED ARRAY:

2 14 21 24 37 58 68 71 75 89
89 90 92 92 105 106 123 136 136 158
176 179 189 194 198 214 225 233 238 251
273 285 289 301 329 338 339 345 378 382
384 426 432 466 467 479 483 484 489 493
498 501 510 510 539 547 557 561 576 576
580 593 606 612 621 624 634 642 645 654
664 665 666 669 670 677 699 735 738 743
759 773 776 822 839 858 884 891 891 898
901 909 917 919 925 942 951 957 971 980
```

OBSERVATION ANALYSIS

- The quick sort is like the previous sorting techniques. But works on even smaller portions. And instead of prev (index i) and current element (index j), it is called low and high. Elements smaller than the pivot are placed on its left, while larger elements are placed on its right.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

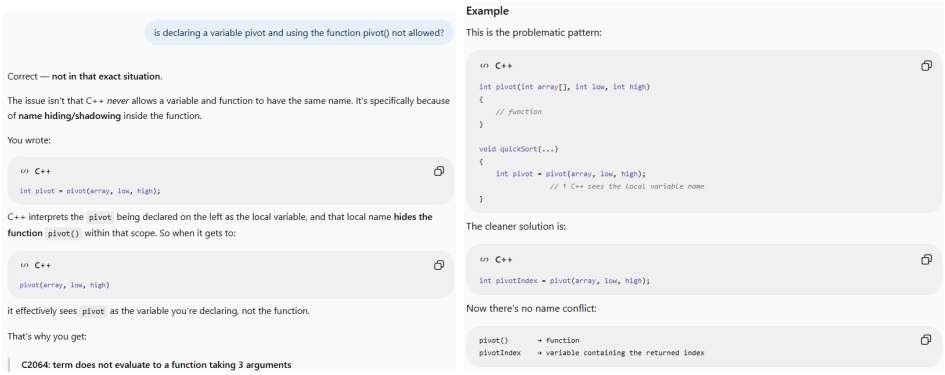
COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use:

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:



is declaring a variable pivot and using the function pivot() not allowed?

Correct — **not in that exact situation**.

The issue isn't that C++ *never* allows a variable and function to have the same name. It's specifically because of **name hiding/shadowing** inside the function.

You wrote:

```
C++
int pivot = pivot(array, low, high);
```

C++ interprets the `pivot` being declared on the left as the local variable, and that local name **hides the function** `pivot()` within that scope. So when it gets to:

```
C++
pivot(array, low, high)
```

it effectively sees `pivot` as the variable you're declaring, not the function.

That's why you get:

> **C2064: term does not evaluate to a function taking 3 arguments**

Example

This is the problematic pattern:

```
C++
int pivot(int array[], int low, int high)
{
    // function
}

void quickSort(...)
{
    int pivot = pivot(array, low, high);
                // ↑ C++ sees the local variable name
}
```

The cleaner solution is:

```
C++
int pivotIndex = pivot(array, low, high);
```

Now there's no name conflict:

```
pivot()      → function
pivotIndex   → variable containing the returned index
```

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.